LAPORAN UJIAN TENGAH SEMESTER

KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK

SISTEM MANAJEMEN ASET LABORATORIUM DENGAN CI/CD DAN CONTAINERIZATION



Disusun oleh:

Nama	: Hafidz Rizqullah Prasetya
NIM	: 24/535493/SV/24243
Kelas	: PL5A1
Dosen Pengampu	: Dr. Muhammad Fakhrurrifqi, S.Kom., M.Cs.
	Galih Malela Damaraji, S.Pd., M.Eng.

PROGRAM STUDI D-IV

TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

Daftar Isi

Daftar Isi **2**

Tujuan Ujian **4**

Dasar Teori **5**
2.1 Arsitektur RESTful API dan Otentikasi Token 5
2.2 Aplikasi Halaman Tunggal dan Klien HTTP 5
2.3 Manajemen Repositori Git dan Proteksi Branch 5
2.4 Otomatisasi Alur Kerja dengan GitHub Actions 5
2.5 Kontainerisasi dan Dockerfile Multi-Stage 6
2.6 Docker Compose 6
2.7 FrankenPHP 6

Hasil dan Pembahasan **7**
3.1 Informasi Repositori 7
3.2 Soal 1: Backend Laravel RESTful API (CPMK 1) 7
3.2.1 Skema Data 7
3.2.2 Otentikasi Token (Soal 1a) 8
3.2.3 Endpoint dan Kode Status (Soal 1b dan 1c) 8
3.2.4 Koleksi Postman (Soal 1d) 8
3.2.5 Feature Test (Soal 1e) 9
3.3 Soal 2: Frontend Vue.js dan Axios (CPMK 1) 9
3.3.1 Alamat API dari Variabel Lingkungan (Soal 2c) 9
3.3.2 Halaman Login dan Penyimpanan Token (Soal 2a) 12
3.3.3 Halaman CRUD Aset (Soal 2b) 12
3.4 Soal 3: Repositori GitHub dan CI (CPMK 2) 17
3.4.1 Repositori dan Struktur Folder (Soal 3a) 17
3.4.2 Branch dan Pull Request (Soal 3b) 17
3.4.3 Branch Protection (Soal 3b) 17
3.4.4 Workflow CI (Soal 3c) 19
3.5 Soal 4: Continuous Deployment ke GHCR (CPMK 3) 21
3.5.1 Deploy Hanya untuk Push ke Main (Soal 4a) 21
3.5.2 Nilai Rahasia di GitHub Secrets (Soal 4b) 22
3.5.3 Image ke GHCR dengan Tag Commit (Soal 4c dan 4d) 23
3.6 Soal 5: Dockerfile Backend dan Frontend (CPMK 4 dan CPMK 5) 26
3.6.1 Dockerfile Backend (Soal 5a) 26
3.6.2 Dockerfile Frontend Multi-Stage (Soal 5b) 26
3.6.3 .dockerignore dan Nilai Rahasia (Soal 5c dan 5d) 27
3.7 Soal 6: Docker Compose (CPMK 6 dan CPMK 7) 28

3.7.1 Tiga Service dalam Satu Berkas (Soal 6a) 28
3.7.2 Koneksi Lewat Nama Service (Soal 6b) 29
3.7.3 CRUD dari Peramban (Soal 6c) 30
3.7.4 Data Bertahan di Named Volume (Soal 6d) 30
3.7.5 Kredensial dari .env (Soal 6e) 30
3.8 Soal 7: Backend dengan FrankenPHP (Nilai Tambahan) 31
3.9 Alur Data dari Vue ke Laravel ke Basis Data 32

Kendala dan Solusi **34**

Kesimpulan **36**

Daftar Pustaka **37**

Tujuan Ujian

Sesuai lembar soal Ujian Tengah Semester mata kuliah *Konstruksi dan Evolusi Perangkat Lunak* Semester Gasal 2026/2027, pengerjaan ujian ini bertujuan agar mahasiswa mampu:

1. Membuat backend Laravel bertema mandiri berupa RESTful API dengan login dan logout berbasis token (Laravel Sanctum), CRUD lengkap yang tersimpan di basis data, kode status 201, 422, dan 401, koleksi Postman, serta minimal dua *feature test* (soal 1, CPMK 1).

2. Membuat frontend Vue.js yang memakai API tersebut melalui Axios: login yang menyimpan token, halaman CRUD, dan alamat API yang diatur lewat variabel lingkungan (soal 2, CPMK 1).

3. Mengelola kode di repositori publik `uts-p1-535493` dengan folder `backend/` dan `frontend/`, branch `main`, `dev`, dan branch fitur, proteksi branch yang mewajibkan Pull Request dan status check, serta `ci.yml` dengan job test backend dan build frontend (soal 3, CPMK 2).

4. Menambahkan job deploy yang hanya berjalan pada push ke `main`, menyimpan nilai rahasia di GitHub Secrets, dan mengunggah image backend serta frontend ke `ghcr.io` dengan tag nomor commit (soal 4, CPMK 3).

5. Menyusun Dockerfile backend dengan *layer caching* Composer dan Dockerfile frontend *multi-stage* berbasis Nginx tanpa `node_modules`, lengkap dengan `.dockerignore` dan tanpa nilai rahasia (soal 5, CPMK 4 dan CPMK 5).

6. Menjalankan seluruh sistem (basis data, backend, frontend) dengan satu perintah `docker compose up -d --build`, dengan koneksi antarservice lewat nama service, data di *named volume*, dan kredensial dari `.env` yang tidak di-commit (soal 6, CPMK 6 dan CPMK 7).

7. Menjalankan backend Laravel dengan FrankenPHP sebagai server aplikasi (soal 7, nilai tambahan).

Dasar Teori

2.1 Arsitektur RESTful API dan Otentikasi Token

REST (*Representational State Transfer*) memakai metode HTTP standar untuk membaca dan mengubah sumber daya. Layanan REST bersifat *stateless*: setiap permintaan membawa sendiri identitas pengirimnya, tanpa sesi yang disimpan di memori peladen [1].

Laravel Sanctum menerbitkan *personal access token* setelah kredensial pengguna diverifikasi. Peladen menyimpan hash token di basis data, sedangkan klien menyimpan teks aslinya dan mengirimkannya pada tajuk `Authorization: Bearer <token>`. Kode status HTTP menjadi bagian dari kontrak API: 201 ketika data dibuat, 422 ketika validasi gagal, dan 401 ketika token tidak ada atau tidak sah [1].

2.2 Aplikasi Halaman Tunggal dan Klien HTTP

Vue.js 3 membangun antarmuka dari komponen reaktif, sehingga tampilan diperbarui tanpa memuat ulang halaman [2]. Axios menghubungkan komponen dengan API. *Request interceptor* menyisipkan token dari `localStorage` ke setiap permintaan, dan *response interceptor* membersihkan sesi ketika peladen membalas 401. Alamat API dibaca dari variabel lingkungan agar tidak ditulis mati di komponen [3].

2.3 Manajemen Repositori Git dan Proteksi Branch

Branch `main` menampung kode yang siap rilis, `dev` menampung hasil integrasi, dan setiap pekerjaan baru dibuat di branch fitur lalu digabung lewat Pull Request. *Branch protection rules* di GitHub menolak push langsung dan mewajibkan status check lulus sebelum merge [4].

2.4 Otomatisasi Alur Kerja dengan GitHub Actions

GitHub Actions menjalankan *workflow* berformat YAML di `.github/workflows/` sebagai respons terhadap kejadian di repositori, misalnya push atau Pull Request. Job yang

tidak saling bergantung berjalan paralel, sedangkan job dengan `needs` dan `if` hanya berjalan setelah syaratnya terpenuhi [5].

2.5 Kontainerisasi dan Dockerfile Multi-Stage

Docker mengemas aplikasi beserta dependensinya ke dalam image yang berjalan sama di mesin mana pun. Setiap instruksi Dockerfile menghasilkan lapisan yang di-*cache*. Dengan menyalin manifes dependensi sebelum kode sumber, langkah instalasi dependensi tidak diulang selama manifesnya tidak berubah. *Multi-stage build* memisahkan tahap kompilasi dari tahap penyajian, sehingga image akhir hanya berisi hasil build [6, 9].

2.6 Docker Compose

Docker Compose mendefinisikan beberapa container dalam satu berkas `docker-compose.yml`. Setiap service masuk ke jaringan yang sama dan dapat saling memanggil dengan nama service sebagai nama host. Data yang harus bertahan disimpan di *named volume* yang dikelola Docker dan tidak ikut terhapus oleh `docker compose down`. Nilai seperti `${DB_PASSWORD}` di dalam berkas compose diambil dari berkas `.env` di folder yang sama [7].

2.7 FrankenPHP

FrankenPHP adalah server aplikasi PHP yang dibangun di atas peladen web Caddy. PHP berjalan langsung di dalam proses peladen, sehingga tidak perlu `php artisan serve` maupun pasangan PHP-FPM dan Nginx. Image resminya menyediakan perintah `frankenphp php-server` untuk menyajikan folder `public/` Laravel [8].

Hasil dan Pembahasan

3.1 Informasi Repositori

Seluruh kode berada di repositori publik `https://github.com/hafidzrizqullahpr asetya/uts-p1-535493`. Isinya:

- `backend/`: Laravel 13, Dockerfile FrankenPHP, `docker/start.sh`, *feature test*, dan koleksi Postman `backend/postman/uts_kepl_asset_api.postman_collection.json`.
- `frontend/`: Vue 3 (Vite) dan Axios, Dockerfile *multi-stage*, serta `nginx.conf`.
- `docker-compose.yml` dan `.env.example` di akar repositori.
- `.github/workflows/ci.yml`.

Seluruh tangkapan layar diambil di laptop pengerjaan (Linux Mint, Cinnamon) dari terminal `gnome-terminal` dan peramban Brave. Port 8000 di laptop ini sudah dipakai aplikasi lain, sehingga backend dari Docker Compose dibuka di port 8001 melalui `BACKEND_PORT=8001` di berkas `.env`. Frontend dibuka di `http://localhost:8080`.

3.2 Soal 1: Backend Laravel RESTful API (CPMK 1)

Tema yang dipilih adalah Sistem Manajemen Aset Laboratorium untuk mencatat peralatan di laboratorium Departemen Teknik Elektro dan Informatika SV UGM.

3.2.1 Skema Data

Sumber daya utama adalah `Asset` dengan atribut nama, kode inventaris unik, kategori, status, nilai perolehan, dan lokasi (Listing 1). Model `Asset` memakai `$fillable` untuk mencegah *mass assignment*.

```
Schema::create('assets', function (Blueprint $table) {
    $table->id();
    $table->string('name');
```

```
    $table->string('code')->unique();
    $table->string('category');
    $table->string('status')->default('tersedia');
    $table->decimal('price', 15, 2);
    $table->string('location')->nullable();
    $table->timestamps();
});
```

Listing 1: Migrasi tabel assets.

3.2.2 Otentikasi Token (Soal 1a)

`POST /api/login` memeriksa email dan kata sandi, lalu menerbitkan token dengan `createToken()`. Rute lain berada di dalam middleware `auth:sanctum`, sehingga hanya dapat diakses dengan token. `POST /api/logout` mencabut token yang sedang dipakai.

3.2.3 Endpoint dan Kode Status (Soal 1b dan 1c)

Seluruh permintaan dan balasan berformat JSON. Daftar endpoint beserta kode status ada di Tabel .1.

Table .1: Daftar endpoint API.

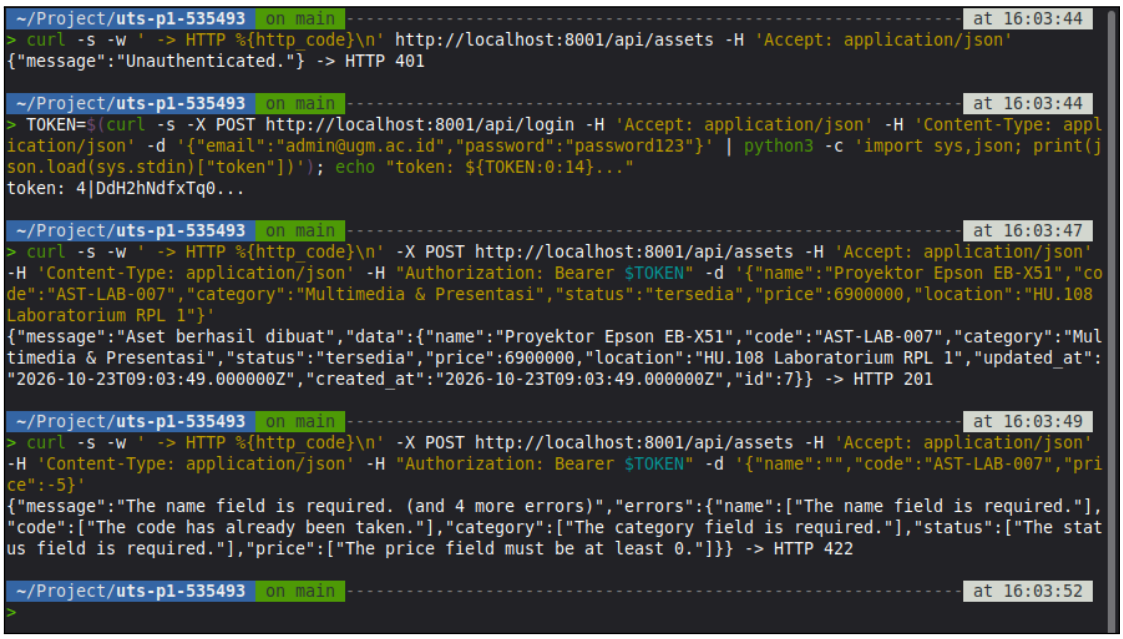
Metode dan rute	Fungsi	Kode status
`POST /api/login`	Login, menerbitkan token	200, 401, 422
`GET /api/me`	Data pengguna yang login	200, 401
`POST /api/logout`	Mencabut token	200, 401
`GET /api/assets`	Tampil semua aset	200, 401
`GET /api/assets/{id}`	Tampil satu aset	200, 401, 404
`POST /api/assets`	Tambah aset	201, 401, 422
`PUT /api/assets/{id}`	Ubah aset	200, 401, 422
`DELETE /api/assets/{id}`	Hapus aset	200, 401

Gambar 1 memperlihatkan pengujian langsung ke backend yang berjalan di Docker Compose. Tanpa token, API membalas 401. Setelah login, penambahan aset yang valid dibalas 201, sedangkan data yang kosong dan kode yang sudah terpakai dibalas 422 beserta pesan untuk setiap kolom.

3.2.4 Koleksi Postman (Soal 1d)

Koleksi Postman disimpan di `backend/postman/uts_kepl_asset_api.postman_collection.json`. Request di dalamnya disusun agar dapat dijalankan berurutan:

1. Login menyimpan token ke variabel koleksi `token`.



Gambar 1: Pengujian kode status 401, 201, dan 422 dengan curl di terminal.

2. Request tambah aset menyimpan `id` hasilnya ke variabel `assetId`, yang kemudian dipakai oleh request tampil satu, ubah, dan hapus.

3. Request uji 422 dan 401 dijalankan setelah CRUD.

4. Logout ada di akhir, diikuti satu request yang memastikan token lama ditolak.

Setiap request memiliki skrip tes yang memeriksa kode status yang diharapkan. Koleksi dijalankan dengan Newman, *runner* baris perintah resmi dari Postman. Hasilnya 11 request dan 11 asersi tanpa kegagalan (Gambar 2).

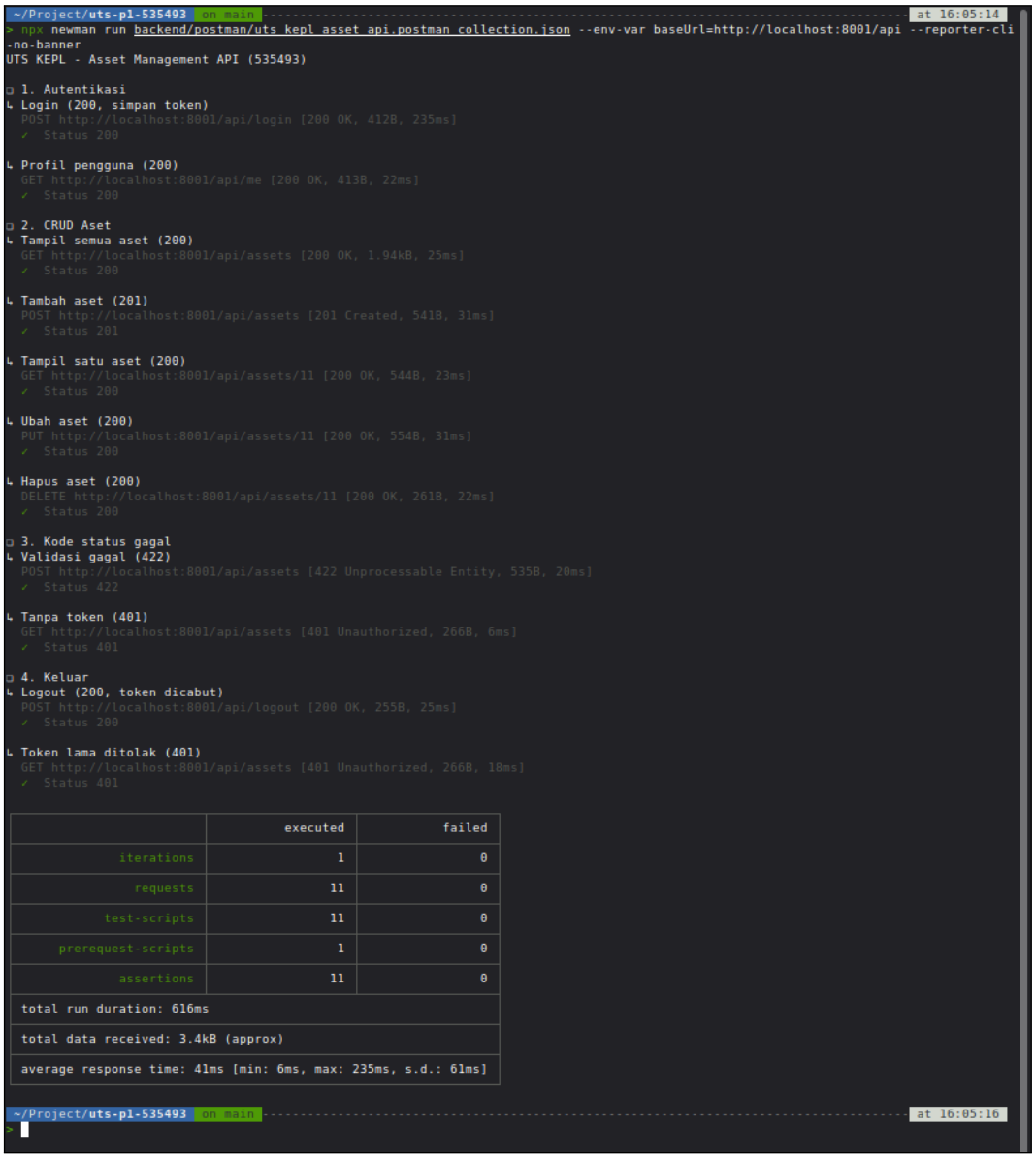
3.2.5 Feature Test (Soal 1e)

Pengujian otomatis ada di `tests/Feature/AuthApiTest.php` (4 tes) dan `tests/Feature/AssetApiTest.php` (7 tes). Keduanya mencakup login berhasil dan gagal, logout, akses tanpa token, serta setiap operasi CRUD termasuk validasi 422. Hasil `php artisan test` ada pada Gambar 3: 13 tes dengan 47 asersi lulus.

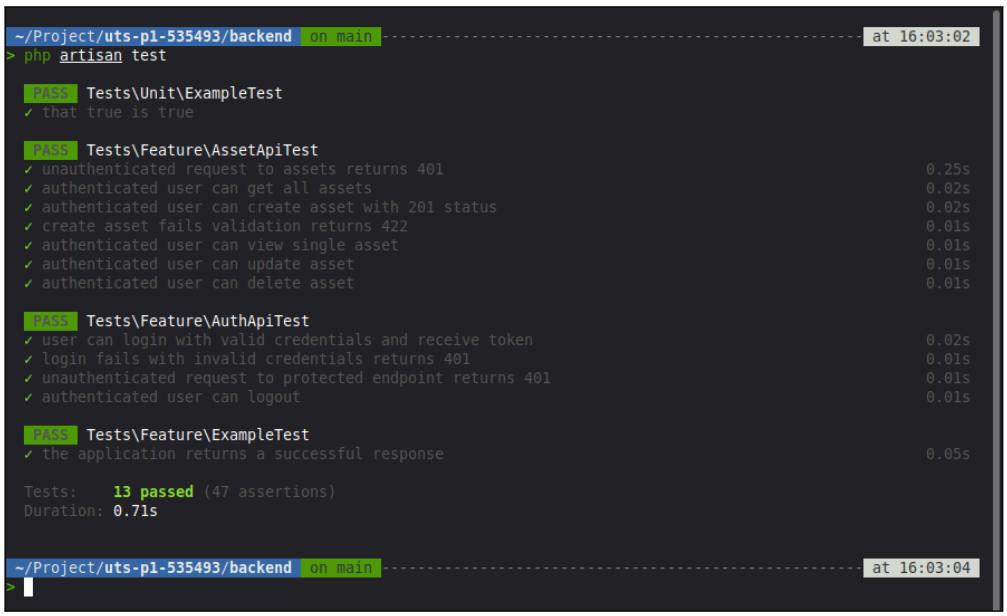
3.3 Soal 2: Frontend Vue.js dan Axios (CPMK 1)

3.3.1 Alamat API dari Variabel Lingkungan (Soal 2c)

Alamat API dibaca dari `import.meta.env.VITE_API_BASE_URL` (Listing 2). Saat pengembangan, nilainya diambil dari `frontend/.env`. Saat image dibangun, nilainya dikirim sebagai *build argument*; di Docker Compose nilainya `/api`, sehingga halaman login menampilkan “Server API /api” (Gambar 4).



Gambar 2: Koleksi Postman dijalankan dengan Newman terhadap backend di Docker Compose.



Gambar 3: Hasil `php artisan test` di terminal.

```
import axios from 'axios';

const apiBaseUrl = import.meta.env.VITE_API_BASE_URL || 'http://localhost:8000/api';

const apiClient = axios.create({
  baseURL: apiBaseUrl,
  headers: {
    'Accept': 'application/json',
    'Content-Type': 'application/json',
  },
});

apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('auth_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('auth_token');
      localStorage.removeItem('auth_user');
      window.dispatchEvent(new CustomEvent('auth:unauthorized'));
    }
    return Promise.reject(error);
```

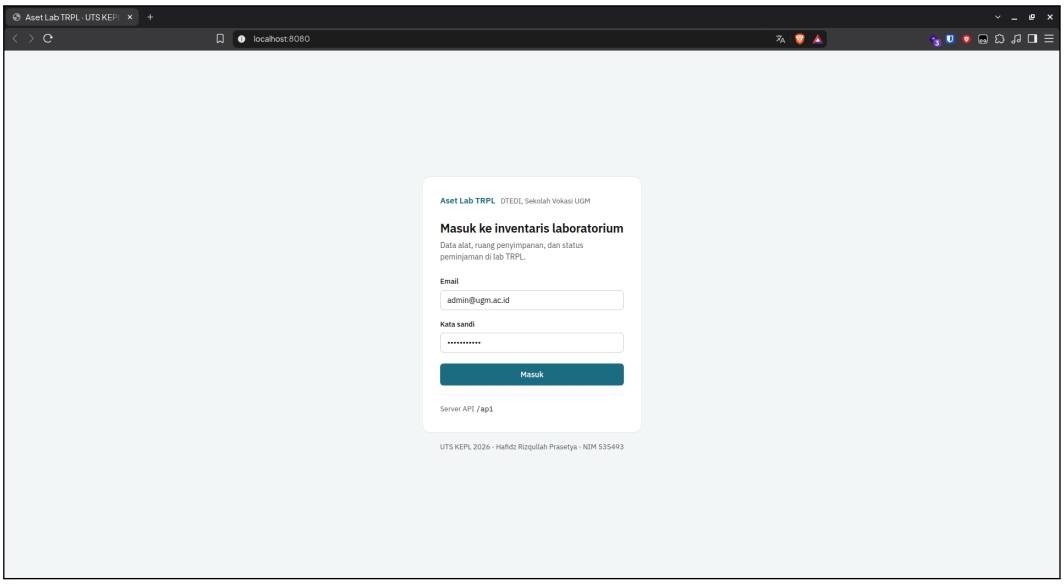
```
  }
);

export default apiClient;
export { apiBaseUrl };
```

Listing 2: Klien Axios di frontend/src/api.js.

3.3.2 Halaman Login dan Penyimpanan Token (Soal 2a)

Formulir login mengirim email dan kata sandi ke `POST /api/login`. Token dari balasan disimpan di `localStorage` dengan kunci `auth_token`, lalu disisipkan oleh *interceptor* pada setiap permintaan berikutnya.

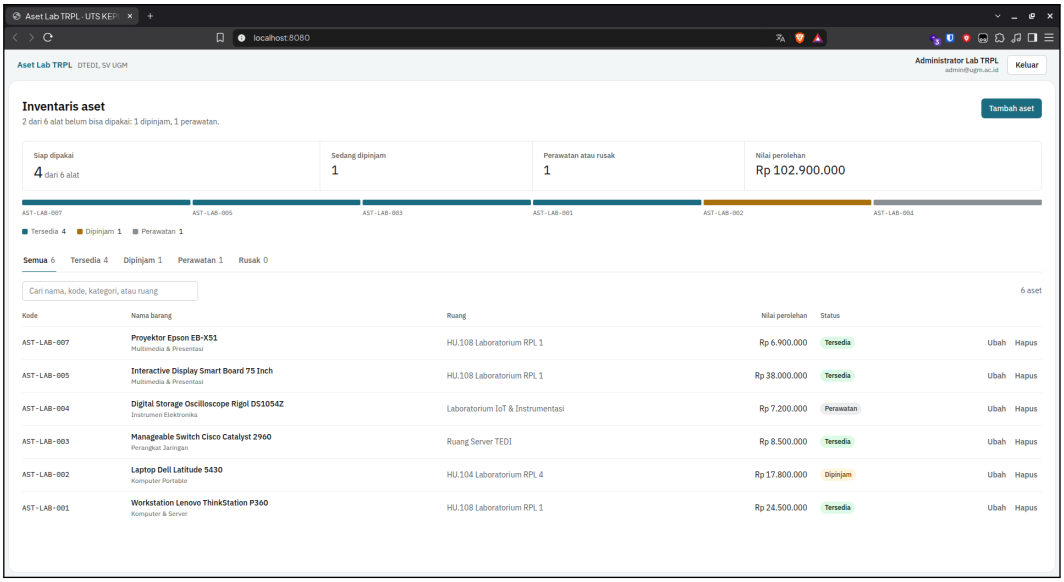


Gambar 4: Halaman login di `http://localhost:8080` (frontend dari Docker Compose).

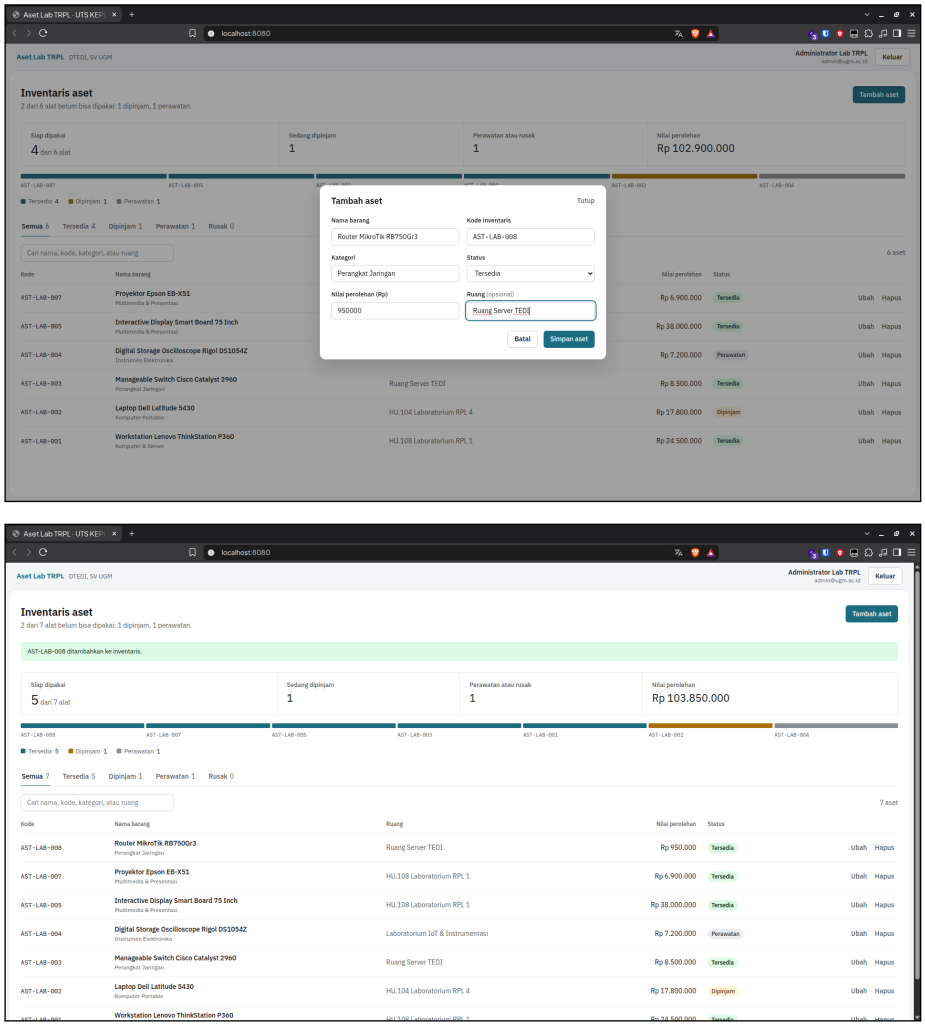
3.3.3 Halaman CRUD Aset (Soal 2b)

Setelah login, halaman inventaris menampilkan ringkasan jumlah alat menurut status, total nilai perolehan, strip kesiapan per kode aset, tab status, kolom pencarian, dan tabel aset (Gambar 5). Tambah dan ubah dilakukan lewat modal, sedangkan hapus meminta konfirmasi lebih dulu. Gambar 6 sampai 8 merekam ketiga operasi tersebut di peramban, dengan data yang benar-benar tersimpan di PostgreSQL (dibuktikan pada Subbab 3.7).

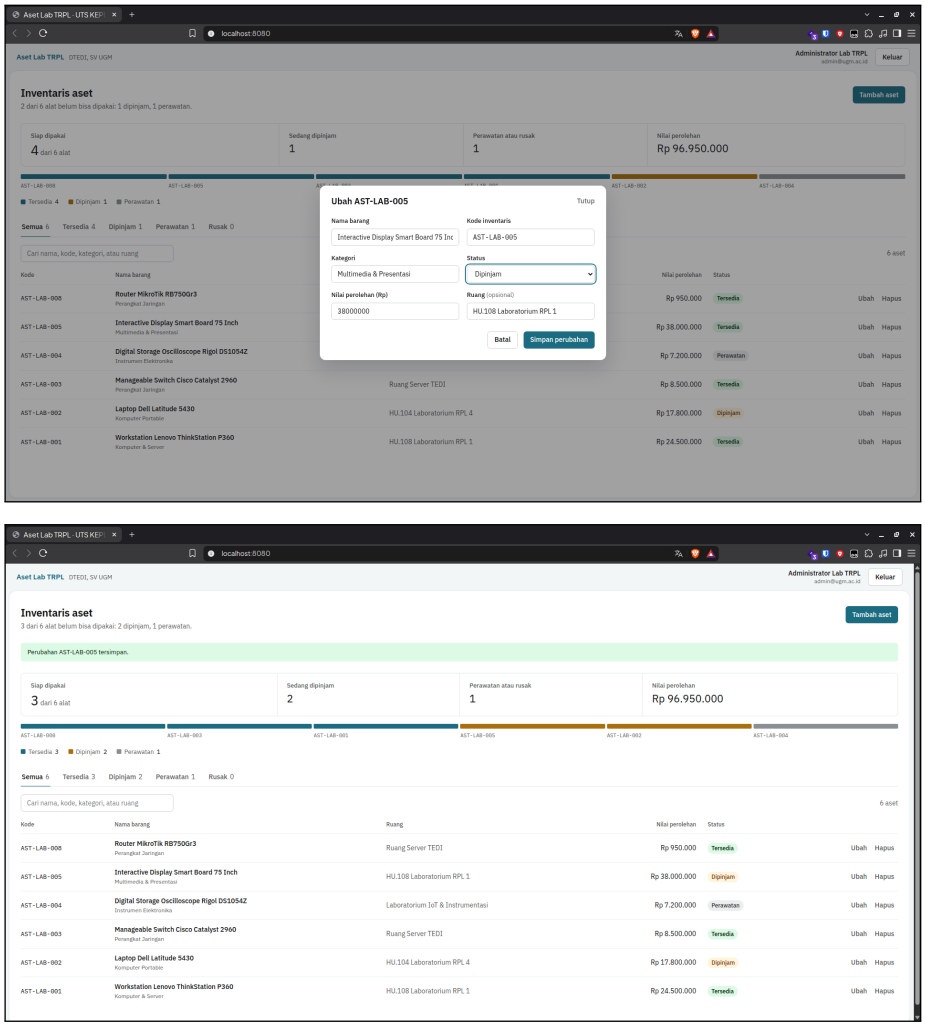
Pada jendela selebar ponsel, ringkasan menjadi dua kolom dan setiap baris tabel menjadi kartu (Gambar 9).



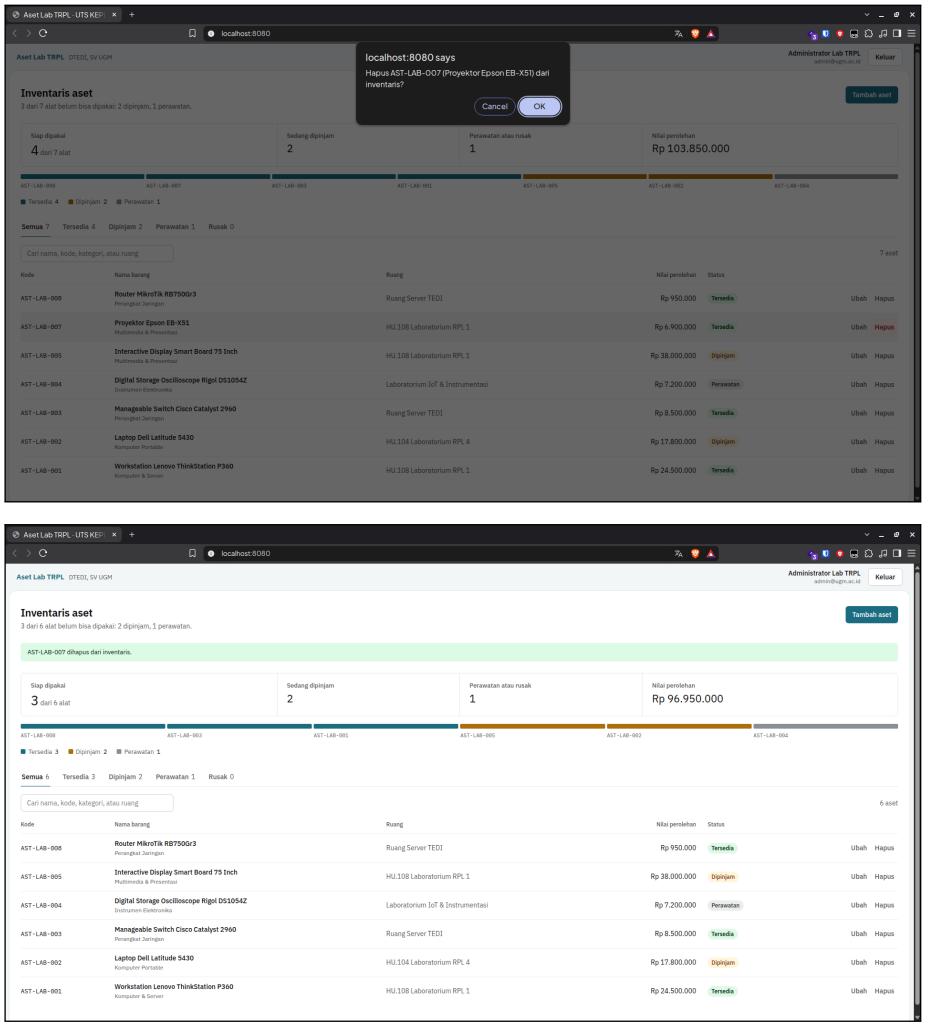
Gambar 5: Halaman inventaris setelah login.



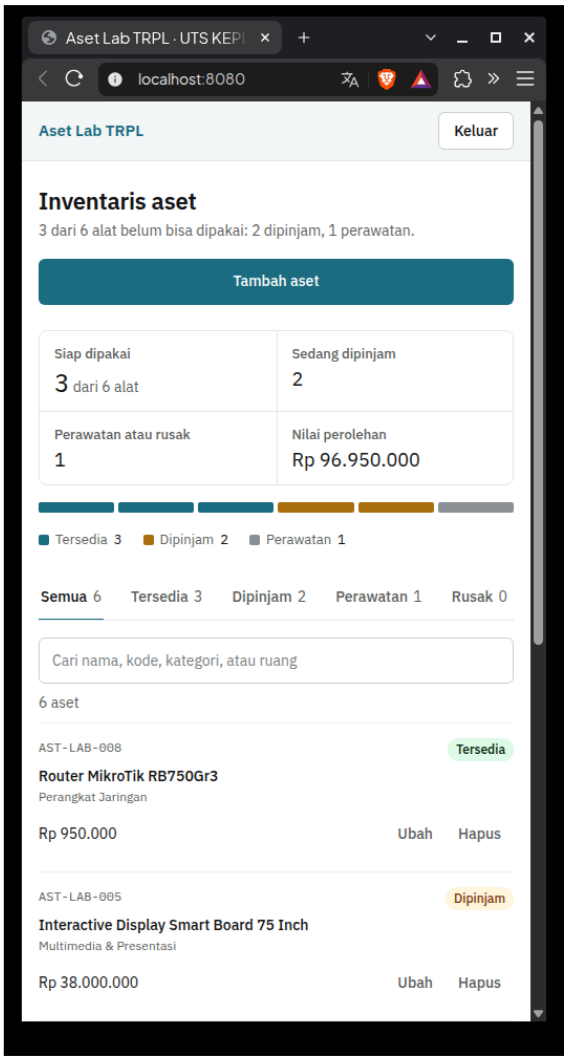
Gambar 6: Tambah aset AST-LAB-008: modal diisi (atas) dan aset muncul di tabel setelah disimpan (bawah).



Gambar 7: Ubah status AST-LAB-005 menjadi Dipinjam: modal ubah (atas) dan hasilnya (bawah).



Gambar 8: Hapus AST-LAB-007: konfirmasi peramban (atas) dan tabel setelah aset dihapus (bawah).

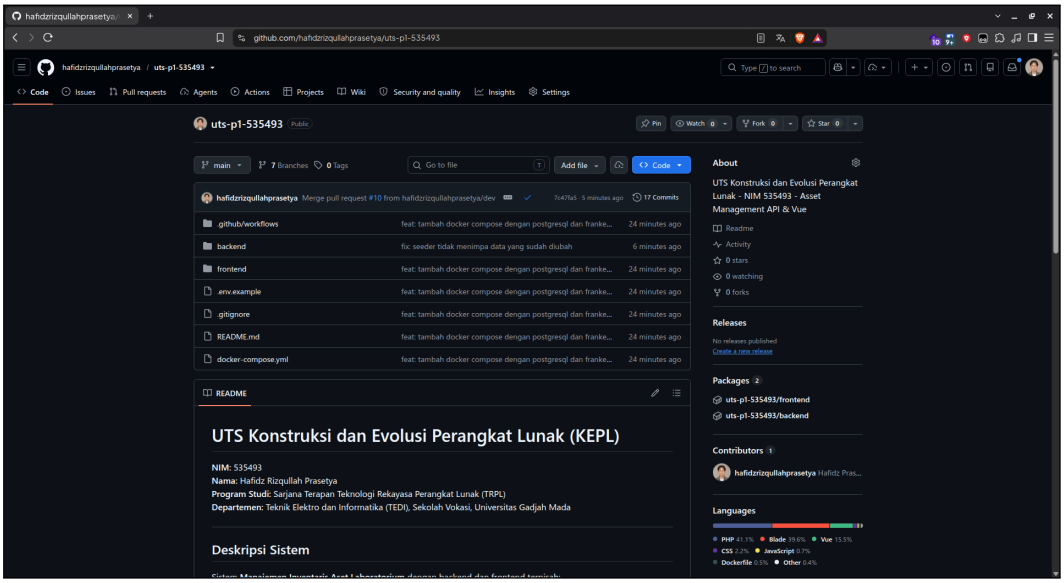


Gambar 9: Halaman inventaris pada jendela peramban selebar ponsel.

3.4 Soal 3: Repositori GitHub dan CI (CPMK 2)

3.4.1 Repositori dan Struktur Folder (Soal 3a)

Repositori publik `uts-p1-535493` berada di akun GitHub pribadi dan berisi folder `backend/` dan `frontend/` (Gambar 10).



Gambar 10: Beranda repositori `uts-p1-535493`.

3.4.2 Branch dan Pull Request (Soal 3b)

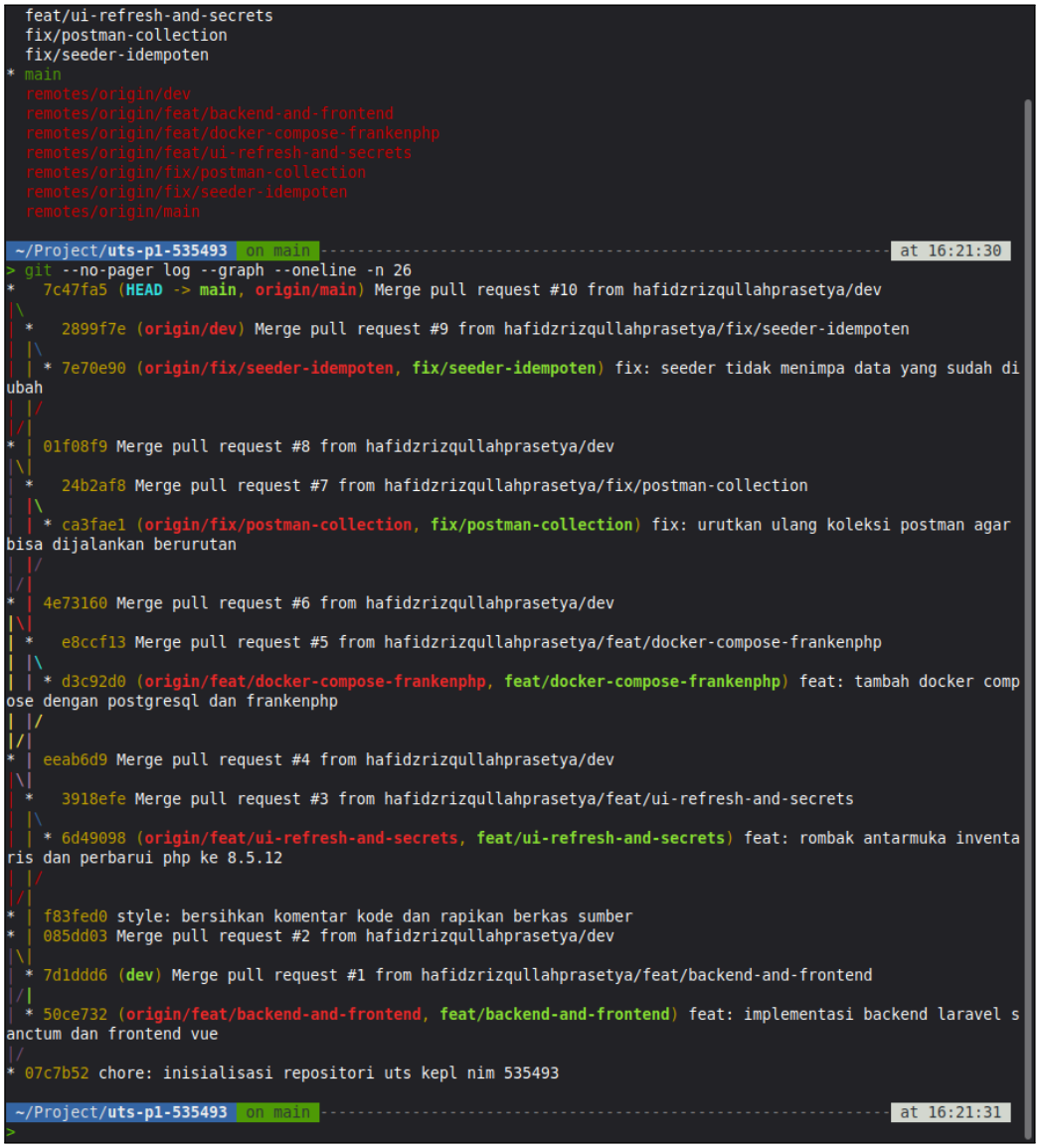
Repositori memiliki branch `main`, `dev`, dan lima branch fitur: `feat/backend-and-frontend`, `feat/ui-refresh-and-secrets`, `feat/docker-compose-frankenphp`, `fix/postman-collection`, dan `fix/seeder-idempoten`. Setiap branch fitur masuk ke `dev` melalui Pull Request, lalu `dev` masuk ke `main` melalui Pull Request berikutnya (PR #1 sampai #10). Graf riwayatnya ada pada Gambar 11.

Komit `f83fed0` (perapian komentar) adalah satu-satunya komit yang masuk ke `main` tanpa Pull Request. Saat komit itu dibuat, aturan proteksi baru mewajibkan status check dan admin repositori masih dapat melewatinya. Aturan kemudian diperketat seperti pada Subbab 3.4.3, dan semua perubahan sesudahnya melalui Pull Request.

3.4.3 Branch Protection (Soal 3b)

Branch `main` dan `dev` memiliki aturan yang sama (Gambar 12):

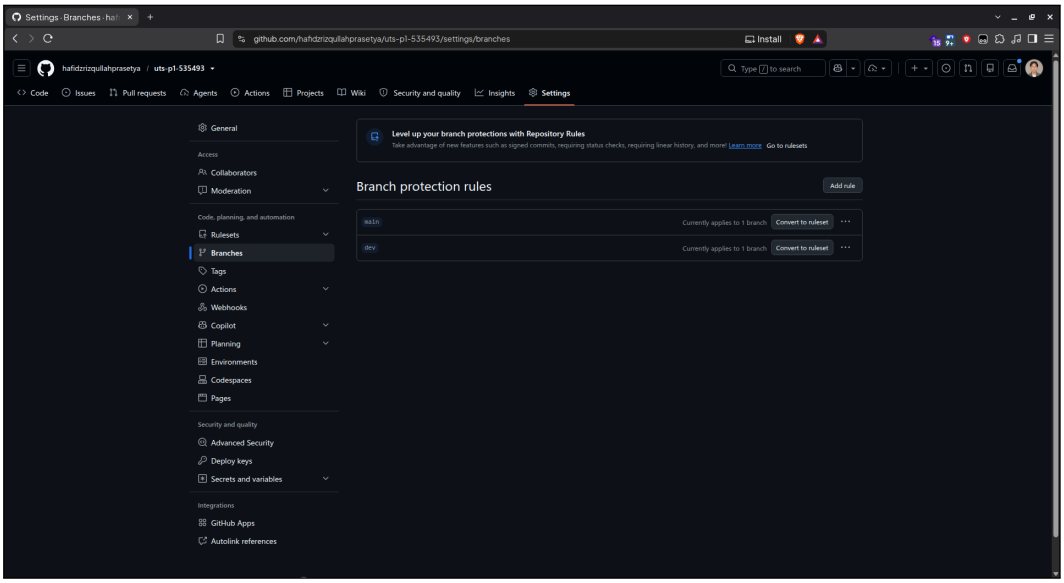
1. *Require a pull request before merging* aktif, sehingga push langsung ditolak (Gambar 13).



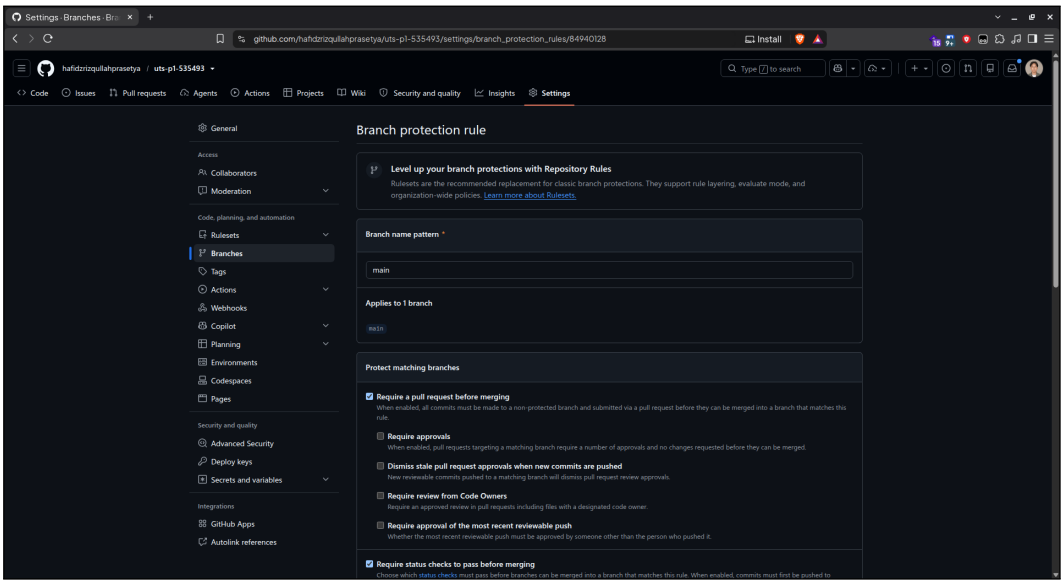
Gambar 11: Daftar branch dan graf komit di terminal.

2. *Require status checks to pass before merging* aktif dengan `Backend Test` dan `Frontend Build` sebagai status check wajib, dan branch harus mutakhir sebelum merge (Gambar 14).

3. *Do not allow bypassing the above settings* aktif, sehingga aturan juga berlaku bagi admin. *Force push* dan penghapusan branch tidak diizinkan (Gambar 15).



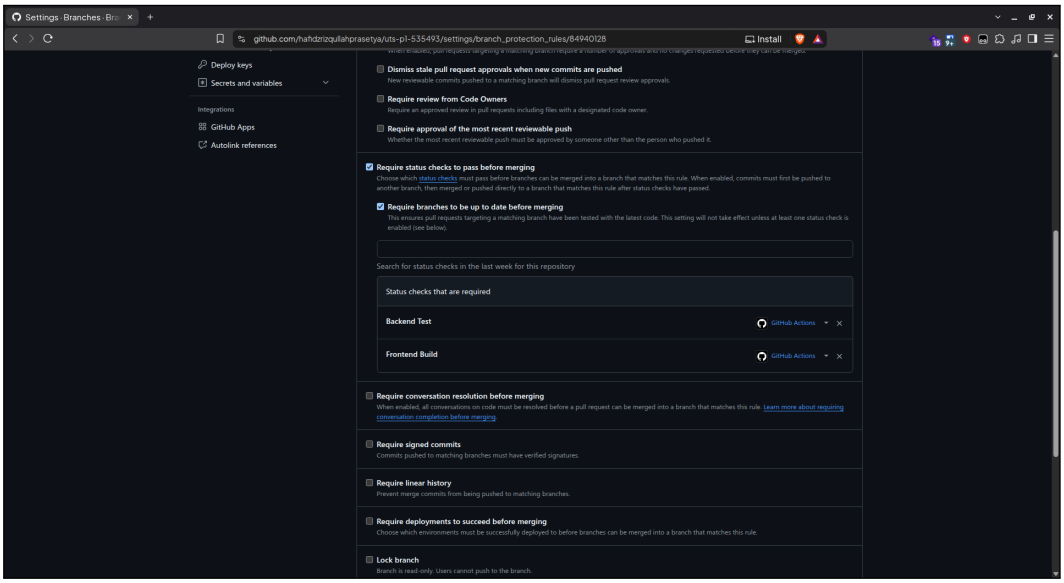
Gambar 12: Halaman Settings → Branches dengan aturan untuk `main` dan `dev`.



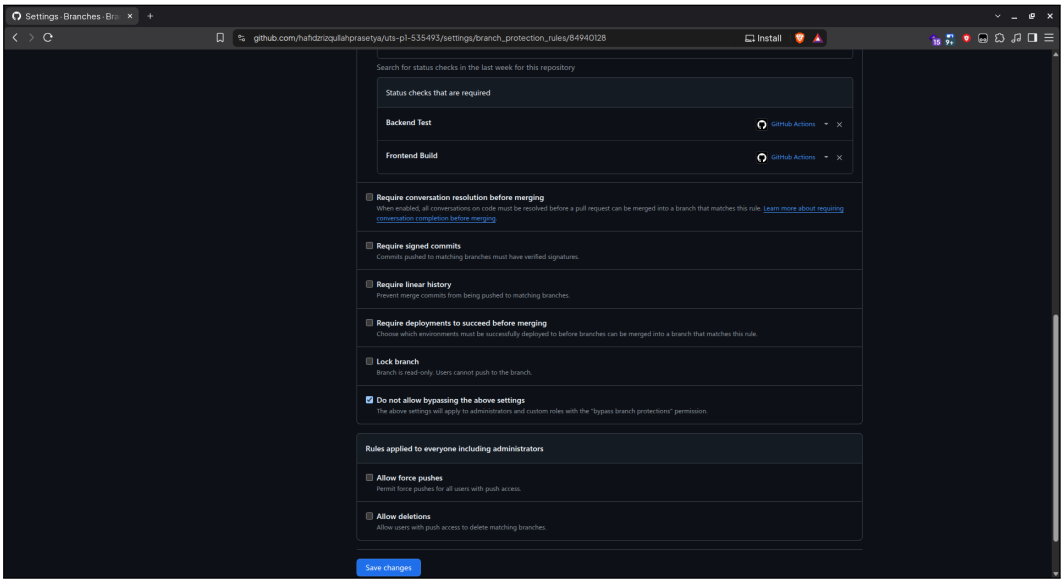
Gambar 13: Aturan `main`: Pull Request wajib sebelum merge.

3.4.4 Workflow CI (Soal 3c)

Berkas `.github/workflows/ci.yml` berjalan pada push dan Pull Request ke `main` dan `dev`, dengan dua job paralel:



Gambar 14: Aturan `main`: status check `Backend Test` dan `Frontend Build` wajib lulus.

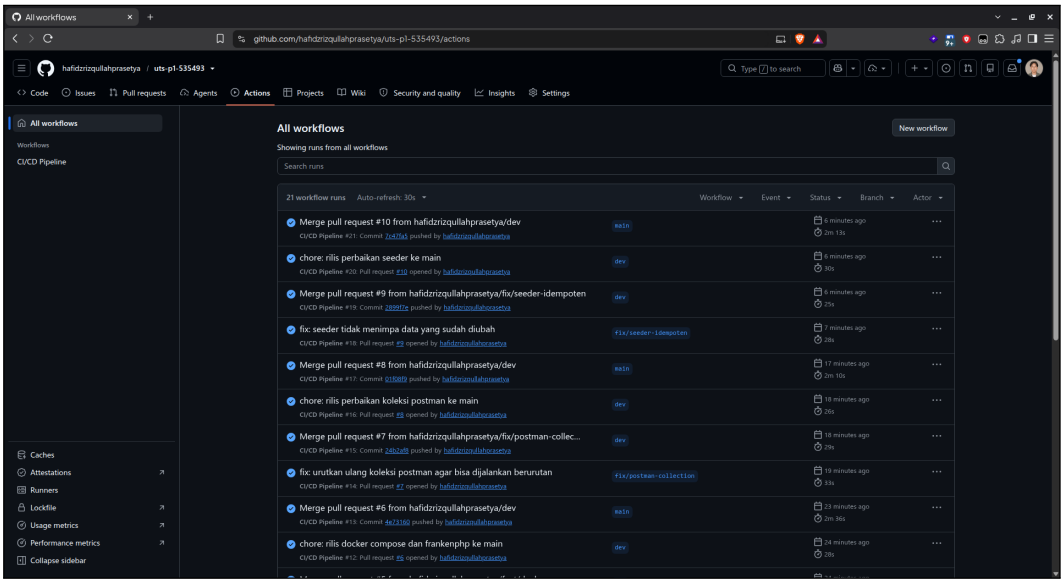


Gambar 15: Aturan `main`: admin tidak dapat melewati aturan, *force push* dan penghapusan branch ditolak.

1. **Backend Test**: menyiapkan PHP 8.5 dan service PostgreSQL 18.6, memasang dependensi Composer, menjalankan migrasi, lalu `php artisan test`.

2. **Frontend Build**: menyiapkan Node.js 20, lalu menjalankan `npm ci` dan `npm run build`.

Seluruh run di tab Actions berstatus hijau (Gambar 16).



Gambar 16: Riwayat run di tab Actions.

3.5 Soal 4: Continuous Deployment ke GHCR (CPMK 3)

3.5.1 Deploy Hanya untuk Push ke Main (Soal 4a)

Job `deploy` bergantung pada kedua job pengujian dan hanya berjalan jika kejadiannya push ke `main` (Listing 3). Pada Pull Request, push ke `dev`, dan branch fitur, workflow berhenti setelah test dan build.

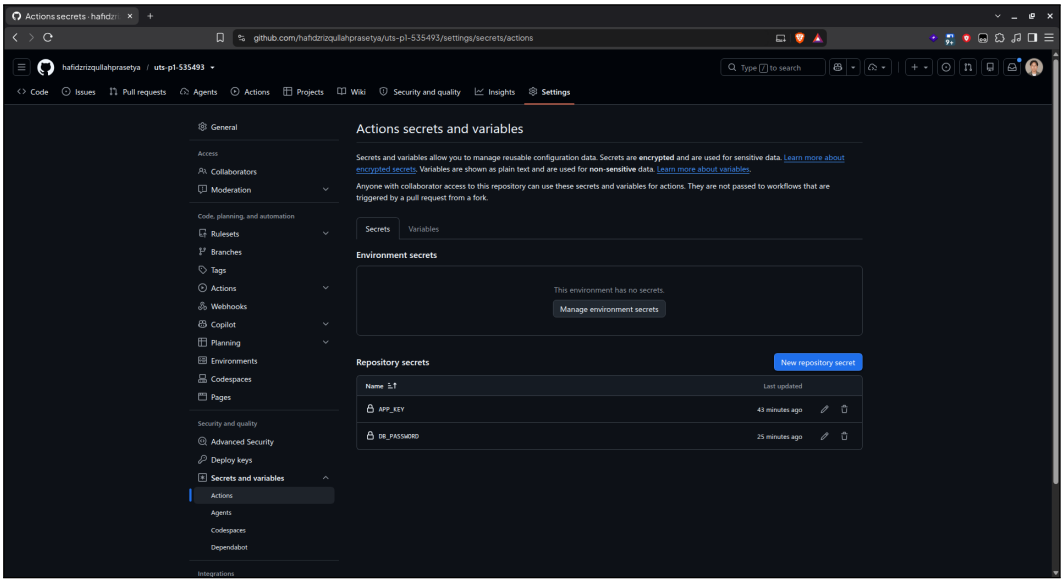
```
deploy:
  name: Deploy to GHCR
  needs: [backend-test, frontend-build]
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Log in to GitHub Container Registry (ghcr.io)
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
```

```
    - uses: docker/setup-buildx-action@v3
    - name: Build and Push Backend Docker Image
      uses: docker/build-push-action@v5
      with:
        context: ./backend
        push: true
        tags: ghcr.io/${{ github.repository }}/backend:${{ github.sha }}
    - name: Build and Push Frontend Docker Image
      uses: docker/build-push-action@v5
      with:
        context: ./frontend
        push: true
        tags: ghcr.io/${{ github.repository }}/frontend:${{ github.sha }}
```

Listing 3: Job deploy pada ci.yml.

3.5.2 Nilai Rahasia di GitHub Secrets (Soal 4b)

`APP_KEY` dan `DB_PASSWORD` disimpan di Settings → Secrets and variables → Actions (Gambar 17) dan tidak ada di dalam repositori. Job `Backend Test` memakai keduanya: `DB_PASSWORD` menjadi kata sandi service PostgreSQL sekaligus kata sandi koneksi Laravel, dan `APP_KEY` menjadi kunci aplikasi saat pengujian (Listing 4). Login ke GHCR memakai `GITHUB_TOKEN` bawaan. Kunci aplikasi tidak dimasukkan ke image, karena image GHCR bersifat publik; kunci diberikan saat container dijalankan, yaitu dari `.env` di Docker Compose.



Gambar 17: Repository secrets `APP_KEY` dan `DB_PASSWORD`.

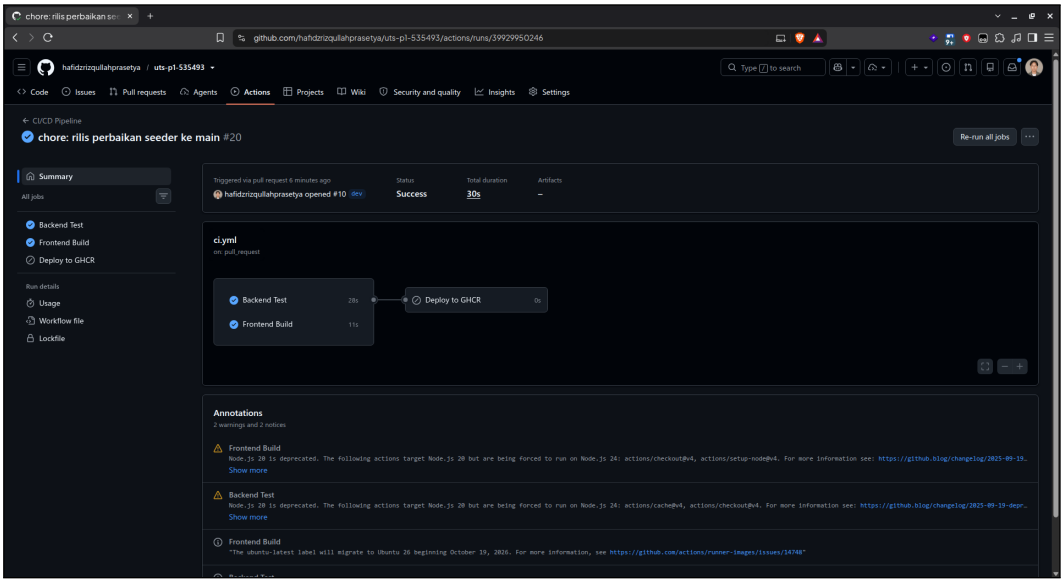
```
backend-test:
  services:
    postgres:
```

```
      image: postgres:18.6-alpine
      env:
        POSTGRES_DB: uts_kepl_test
        POSTGRES_USER: uts_kepl
        POSTGRES_PASSWORD: ${{ secrets.DB_PASSWORD }}
  env:
    APP_KEY: ${{ secrets.APP_KEY }}
    DB_CONNECTION: pgsql
    DB_HOST: 127.0.0.1
    DB_DATABASE: uts_kepl_test
    DB_USERNAME: uts_kepl
    DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
```

Listing 4: Pemakaian GitHub Secrets pada job Backend Test.

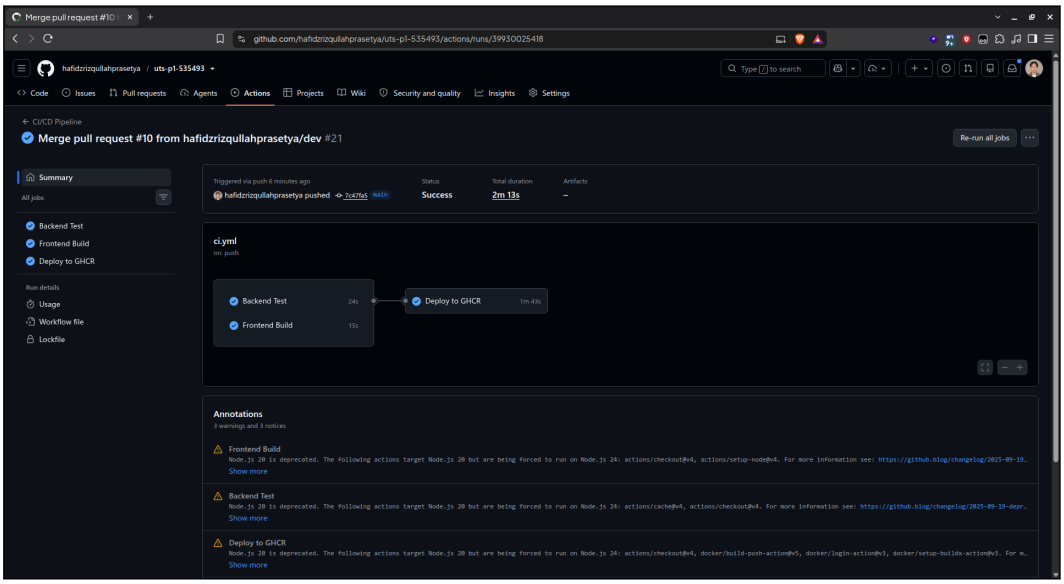
3.5.3 Image ke GHCR dengan Tag Commit (Soal 4c dan 4d)

Gambar 18 adalah run Pull Request #10 dari `dev`: test dan build hijau, sedangkan deploy dilewati. Gambar 19 adalah run push ke `main` setelah Pull Request #10 digabung: ketiga job hijau.

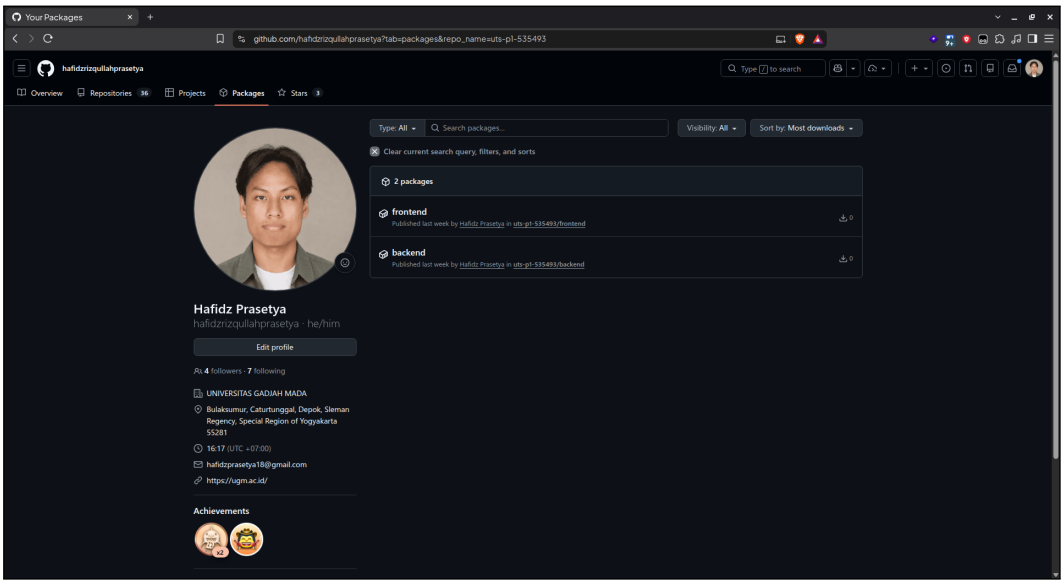


Gambar 18: Run Pull Request dari `dev`: job deploy dilewati.

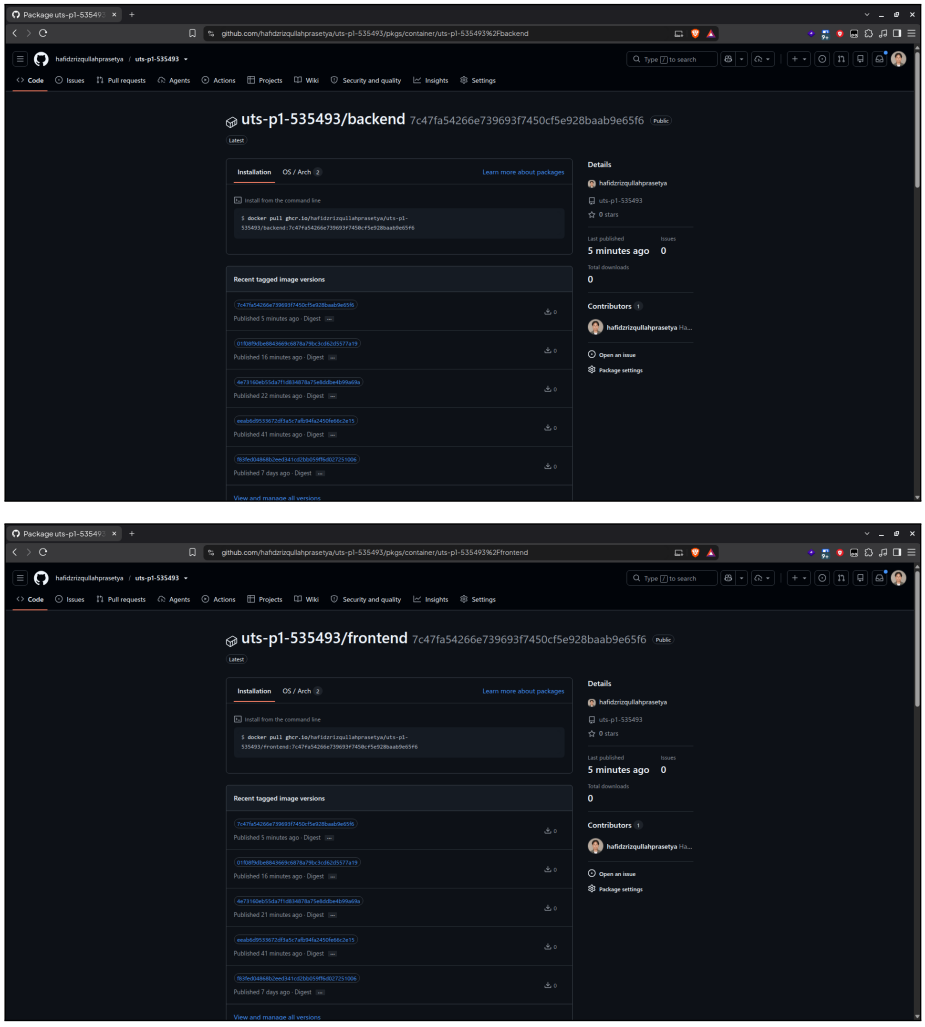
Halaman Packages berisi dua image, `uts-p1-535493/backend` dan `uts-p1-535493/frontend` (Gambar 20). Setiap versi diberi tag hash commit penuh, misalnya `7c47fa5`… untuk komit merge Pull Request #10 (Gambar 21).



Gambar 19: Run push ke `main`: job deploy hijau.



Gambar 20: Halaman Packages berisi image backend dan frontend.



Gambar 21: Versi image backend (atas) dan frontend (bawah) bertag hash commit.

3.6 Soal 5: Dockerfile Backend dan Frontend (CPMK 4 dan CPMK 5)

3.6.1 Dockerfile Backend (Soal 5a)

Image dasar backend adalah `dunglas/frankenphp:1.13.2-php8.5.12-alpine`, dengan versi FrankenPHP dan PHP yang dikunci. `composer.json` dan `composer.lock` disalin dan dipasang sebelum kode aplikasi, sehingga lapisan instalasi Composer dipakai ulang selama manifesnya tidak berubah (Listing 5).

```
FROM dunglas/frankenphp:1.13.2-php8.5.12-alpine

WORKDIR /app

RUN install-php-extensions \
    pdo_pgsql \
    bcmath \
    zip \
    intl

COPY --from=composer:2.8 /usr/bin/composer /usr/bin/composer

COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader --prefer-dist --no-
↪ interaction

COPY . .

RUN composer dump-autoload --optimize \
    && chmod +x docker/start.sh \
    && chown -R www-data:www-data storage bootstrap/cache

EXPOSE 8000

CMD ["docker/start.sh"]
```

Listing 5: backend/Dockerfile.

3.6.2 Dockerfile Frontend Multi-Stage (Soal 5b)

Tahap `builder` memakai `node:20-alpine` untuk `npm ci` dan `npm run build`. Tahap kedua memakai `nginx:1.27-alpine` dan hanya menyalin folder `dist` (Listing 6). Pemeriksaan di dalam image akhir menunjukkan isinya hanya `assets/` dan `index.html`, dengan nol folder `node_modules` (Gambar 24).

```
FROM node:20-alpine AS builder
```

```
2
3  WORKDIR /app
4
5  COPY package*.json ./
6  RUN npm ci
7
8  COPY . .
9  ARG VITE_API_BASE_URL=/api
10 ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
11 RUN npm run build
12
13 FROM nginx:1.27-alpine
14
15 RUN rm -rf /usr/share/nginx/html/*
16 COPY nginx.conf /etc/nginx/conf.d/default.conf
17 COPY --from=builder /app/dist /usr/share/nginx/html
18
19 EXPOSE 80
20
21 CMD ["nginx", "-g", "daemon off;"]
```

Listing 6: frontend/Dockerfile.

3.6.3 .dockerignore dan Nilai Rahasia (Soal 5c dan 5d)

Kedua folder memiliki `.dockerignore` yang mengecualikan `.env`, `.git`, dan folder dependensi (Listing 7). Tidak ada kata sandi, kunci, atau token yang ditulis di kedua Dockerfile.

```
# backend/.dockerignore          # frontend/.dockerignore
vendor                           node_modules
node_modules                     dist
.git                             .git
.env                             .env.local
.phpunit.result.cache            .env.*.local
database/database.sqlite         .env
storage/*.key
```

Listing 7: Isi backend/.dockerignore (kiri) dan frontend/.dockerignore (kanan) digabung.

Tag `latest` pada keluaran `docker compose images` (Gambar 24) adalah nama lokal yang diberikan Docker Compose untuk image hasil *build* di laptop. Image dasar di kedua Dockerfile tetap memakai versi yang jelas, dan image di GHCR bertag hash commit.

3.7 Soal 6: Docker Compose (CPMK 6 dan CPMK 7)

3.7.1 Tiga Service dalam Satu Berkas (Soal 6a)

`docker-compose.yml` di akar repositori berisi service `db` (PostgreSQL 18.6), `backend` (Laravel dengan FrankenPHP), dan `frontend` (Nginx berisi hasil build Vue), seperti pada Listing 8. Backend baru dijalankan setelah *healthcheck* basis data lulus, dan frontend baru dijalankan setelah *healthcheck* `/up` backend lulus.

```
name: uts-kepl-535493

services:
  db:
    image: postgres:18.6-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_DATABASE}
      POSTGRES_USER: ${DB_USERNAME}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10

  backend:
    build: ./backend
    restart: unless-stopped
    environment:
      APP_NAME: "Aset Lab TRPL"
      APP_ENV: production
      APP_DEBUG: "false"
      APP_KEY: ${APP_KEY}
      LOG_CHANNEL: stderr
      DB_CONNECTION: pgsql
      DB_HOST: db
      DB_PORT: 5432
      DB_DATABASE: ${DB_DATABASE}
      DB_USERNAME: ${DB_USERNAME}
      DB_PASSWORD: ${DB_PASSWORD}
    depends_on:
      db:
        condition: service_healthy
    ports:
      - "${BACKEND_PORT:-8000}:8000"
    healthcheck:
```

```
      test: ["CMD", "curl", "-fs", "http://localhost:8000/up"]
      interval: 10s
      timeout: 5s
      retries: 10

  frontend:
    build:
      context: ./frontend
      args:
        VITE_API_BASE_URL: /api
    restart: unless-stopped
    depends_on:
      backend:
        condition: service_healthy
    ports:
      - "${FRONTEND_PORT:-8080}:80"

volumes:
  pgdata:
```

Listing 8: docker-compose.yml.

Gambar 22 adalah keluaran `docker compose ps`: ketiga container berjalan, `db` dan `backend` berstatus *healthy*, dan volume `uts-kepl-535493_pgdata` sudah terbentuk.



Gambar 22: Keluaran `docker compose ps` dan `docker volume ls`.

3.7.2 Koneksi Lewat Nama Service (Soal 6b)

Backend terhubung ke basis data melalui `DB_HOST: db`. Frontend tidak memanggil backend lewat alamat IP laptop. Peramban memanggil `/api` di origin yang sama (`localhost:8080`), lalu Nginx di container frontend meneruskannya ke `http://backend:8000` melalui DNS internal Docker (Listing 9). Log backend pada Gambar 25 mencatat permintaan `GET /api/assets` dari peramban dengan `remote_ip 172.24.0.4`, yaitu alamat container frontend di jaringan Compose.

```
server {
    listen 80;
    server_name localhost;

    resolver 127.0.0.11 valid=10s ipv6=off;
```

```
    set $backend_upstream http://backend:8000;

    location /api/ {
        proxy_pass $backend_upstream;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }
}
```

Listing 9: frontend/nginx.conf.

3.7.3 CRUD dari Peramban (Soal 6c)

Seluruh tangkapan layar frontend pada Subbab soal 2 diambil dari `http://localhost:8080`, yaitu container frontend Docker Compose. Login, tampil, tambah AST-LAB-008, ubah status AST-LAB-005, dan hapus AST-LAB-007 berjalan dari peramban. Isi tabel `assets` di container `db` pada Gambar 23 memuat hasil yang sama: AST-LAB-008 ada, AST-LAB-005 berstatus `dipinjam`, dan AST-LAB-007 sudah tidak ada.

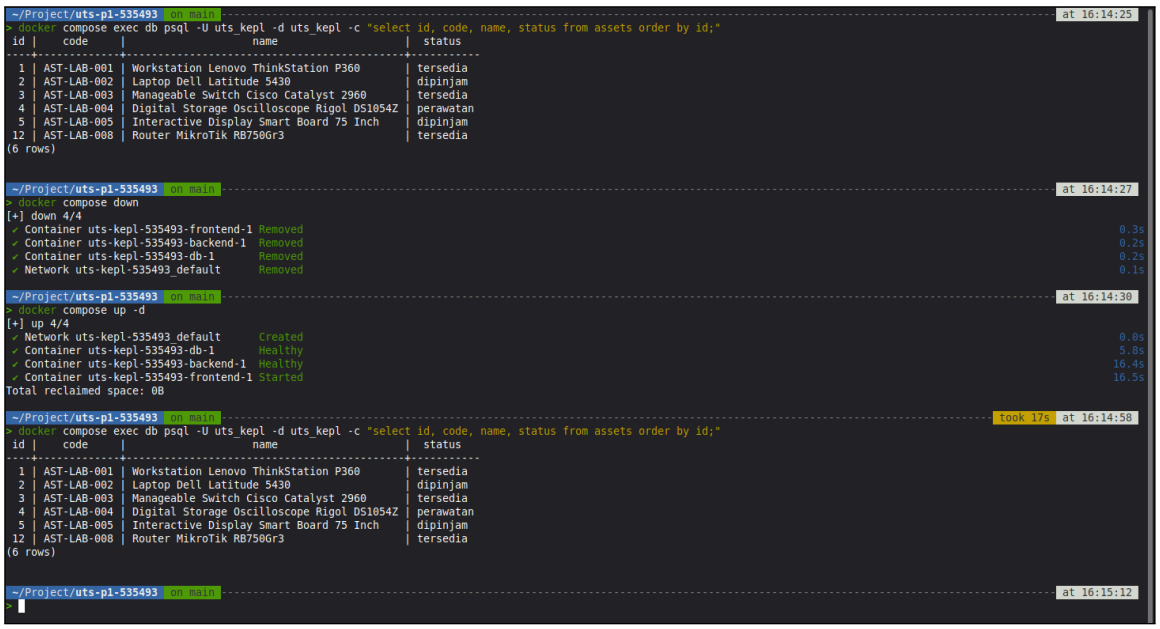
3.7.4 Data Bertahan di Named Volume (Soal 6d)

Data PostgreSQL disimpan di volume `pgdata`. Pengujian pada Gambar 23 membaca tabel `assets`, menjalankan `docker compose down` yang menghapus ketiga container dan jaringannya, menjalankan `docker compose up -d`, lalu membaca tabel lagi. Isi tabel sebelum dan sesudah sama, termasuk aset yang ditambah dan diubah dari peramban.

3.7.5 Kredensial dari .env (Soal 6e)

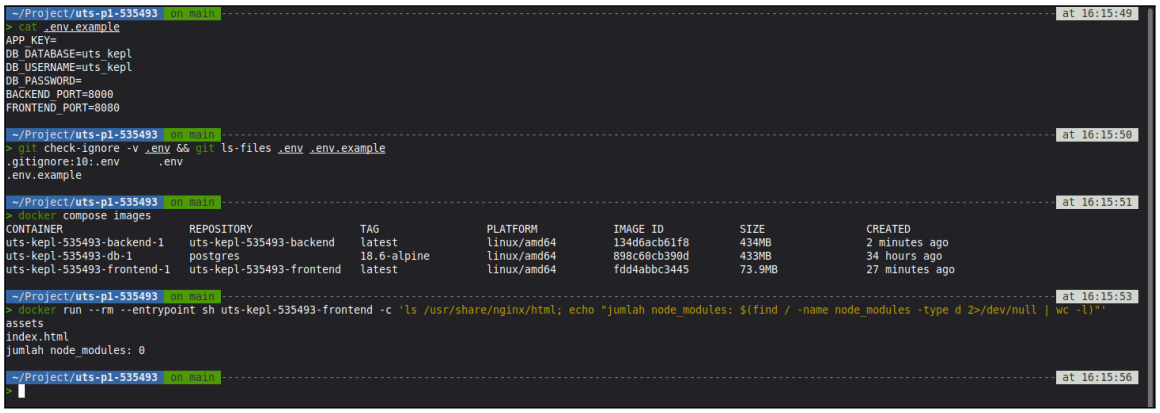
Nama basis data, pengguna, kata sandi, dan `APP_KEY` dibaca dari berkas `.env` di akar repositori. Berkas ini tercantum di `.gitignore` dan tidak di-commit, sedangkan `.env.example` di-commit sebagai contoh dengan nilai rahasia dikosongkan (Gambar 24). Cara menjalankannya:

```
cp .env.example .env
# isi APP_KEY (format base64:...) dan DB_PASSWORD di .env
docker compose up -d --build
```



Gambar 23: Isi tabel `assets` sebelum `docker compose down` dan sesudah `docker compose up -d`.

Listing 10: Menjalankan sistem dengan Docker Compose.



Gambar 24: Isi `.env.example`, bukti `.env` diabaikan Git, daftar image, dan isi image frontend tanpa `node_modules`.

3.8 Soal 7: Backend dengan FrankenPHP (Nilai Tambahan)

Backend tidak lagi memakai `php artisan serve`. Image dasarnya adalah image resmi FrankenPHP, dan container menjalankan `docker/start.sh`: *cache* konfigurasi, migrasi, seeder, lalu `frankenphp php-server` dengan *access log* aktif (Listing 11).

```
#!/bin/sh
```

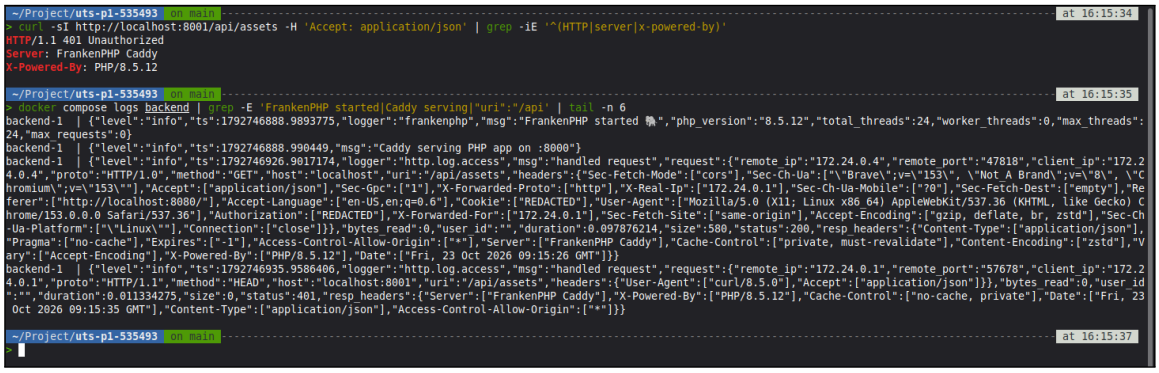
```
set -e

php artisan config:cache
php artisan migrate --force
php artisan db:seed --force

exec frankenphp php-server --root public --listen :8000 --access-log
```

Listing 11: backend/docker/start.sh.

Gambar 25 menunjukkan bahwa respons API membawa tajuk `Server: FrankenPHP Caddy` dan `X-Powered-By: PHP/8.5.12`. Log container backend mencatat "FrankenPHP started" dengan `php_version` 8.5.12, serta `http.log.access` untuk setiap permintaan `/api/assets`.



Gambar 25: Tajuk respons dan log container backend yang menunjukkan permintaan dilayani FrankenPHP.

3.9 Alur Data dari Vue ke Laravel ke Basis Data

Contoh alur ketika pengguna menyimpan aset baru dari peramban:

1. Komponen Vue memanggil `apiClient.post('/assets', form)`. Karena `baseURL` bernilai `/api`, Axios mengirim `POST http://localhost:8080/api/assets` dengan tajuk `Authorization: Bearer <token>` dari `localStorage`.

2. Nginx di container `frontend` mencocokkan `location /api/` dan meneruskan permintaan ke `http://backend:8000/api/assets` lewat jaringan Docker Compose.

3. FrankenPHP di container `backend` meneruskan permintaan ke Laravel. Middleware `auth:sanctum` mencocokkan hash token dengan tabel `personal_access_tokens`, lalu `AssetController@store` memvalidasi data.

4. Eloquent menulis baris baru ke tabel `assets` di PostgreSQL pada host `db`. Data fisiknya berada di volume `pgdata`.

5. Laravel membalas 201 berisi data aset. Balasan kembali melalui Nginx ke peramban, dan Vue menambahkan aset itu ke tabel serta memperbarui angka ringkasan tanpa memuat ulang halaman.

Jika validasi gagal, Laravel membalas 422 dan Vue menampilkan pesannya di bawah kolom terkait. Jika token tidak sah, Laravel membalas 401 dan *interceptor* Axios menghapus sesi lalu mengembalikan pengguna ke halaman login.

Kendala dan Solusi

1. **Komit langsung ke main.** Komit `f83fed0` masuk ke `main` tanpa Pull Request karena aturan awal hanya mewajibkan status check dan admin masih bisa melewatinya. Solusinya, aturan `main` dan `dev` ditambah *Require a pull request before merging* dan *Do not allow bypassing the above settings*. Sejak itu semua perubahan masuk lewat Pull Request.

2. **APP_KEY di GitHub Secrets tidak valid.** Saat `APP_KEY` mulai dibaca dari Secrets, `Backend Test` gagal dengan pesan "Unsupported cipher or incorrect key length". Nilai lama ternyata salah format dan tidak terdeteksi karena sebelumnya secret itu belum pernah dipakai. Solusinya, kunci baru berformat `base64:` dengan panjang 32 byte dibuat ulang lalu disimpan ke Secrets, kemudian job dijalankan ulang.

3. **Port 8000 sudah dipakai.** Di laptop pengerjaan, port 8000 dipakai aplikasi Python (uvicorn) lain. Port backend di `docker-compose.yml` dibuat dapat diatur lewat `BACKEND_PORT` dengan nilai bawaan 8000, dan di laptop ini diisi 8001 lewat `.env`.

4. **Healthcheck bawaan image FrankenPHP.** Image FrankenPHP membawa *healthcheck* yang memeriksa *admin endpoint* di port 2019, sedangkan `php-server` mematikan endpoint tersebut. Akibatnya backend tertahan di status *health: starting*. Solusinya, *healthcheck* di `docker-compose.yml` diganti dengan pemeriksaan rute `/up` milik Laravel, dan frontend menunggu backend *healthy*.

5. **Perubahan data hilang setelah down/up.** Pada uji persistensi pertama, aset baru tetap ada, tetapi status AST-LAB-005 yang diubah menjadi Dipinjam kembali ke Tersedia. Penyebabnya, seeder dijalankan setiap kali container backend start dan memakai `updateOrCreate`, sehingga data seed ditimpa ke nilai awal. Seeder diubah menjadi `firstOrCreate` yang hanya membuat data yang belum ada (Pull Request #9), lalu uji persistensi diulang dan hasilnya sesuai (Gambar 23).

6. **Koleksi Postman tidak bisa dijalankan berurutan.** Versi awal koleksi meletakkan logout sebelum CRUD sehingga token sudah dicabut, dan request ubah serta hapus memakai ID tetap `1` yang ikut menghapus data seed. Koleksi disusun ulang (Pull Request #7): ID diambil dari respons tambah aset, logout dipindah ke akhir, dan setiap

request diberi tes kode status.

Kesimpulan

1. Backend Laravel untuk Sistem Manajemen Aset Laboratorium melindungi seluruh rute privat dengan token Sanctum, menyediakan CRUD aset, dan mengembalikan 201, 422, dan 401 sesuai kondisinya. Perilaku ini dibuktikan oleh 13 *feature test* yang lulus dan 11 request koleksi Postman yang lulus di Newman (CPMK 1).

2. Frontend Vue 3 dengan Axios menangani login, menyimpan token di `localStorage`, dan menjalankan tampil, tambah, ubah, serta hapus dari peramban. Alamat API diatur lewat `VITE_API_BASE_URL` (CPMK 1).

3. Kode dikelola di repositori publik `uts-p1-535493`. Branch `main` dan `dev` mewajibkan Pull Request serta kelulusan `Backend Test` dan `Frontend Build`, dan aturan itu juga mengikat admin (CPMK 2).

4. Job deploy hanya berjalan pada push ke `main`. `APP_KEY` dan `DB_PASSWORD` disimpan di GitHub Secrets, dan image backend serta frontend dipublikasikan ke `ghcr.io` dengan tag hash commit (CPMK 3).

5. Dockerfile backend memasang dependensi Composer sebelum menyalin kode, dan Dockerfile frontend memakai *multi-stage build* sehingga image akhir hanya berisi Nginx dan hasil build (CPMK 4 dan CPMK 5).

6. `docker compose up -d --build` menjalankan PostgreSQL, backend, dan frontend yang saling terhubung lewat nama service. Data bertahan setelah `down` dan `up` karena disimpan di *named volume*, dan kredensial dibaca dari `.env` yang tidak di-commit (CPMK 6 dan CPMK 7).

7. Backend dilayani FrankenPHP 1.13.2 dengan PHP 8.5.12, dibuktikan oleh tajuk respons dan log container (nilai tambahan).

Daftar Pustaka

[1] Laravel LLC, "Laravel Sanctum," 2026. [Daring]. Tersedia: https://laravel.com/docs/sanctum. [Diakses: 23-Okt-2026].

[2] Vue.js Core Team, "Introduction," Vue.js Guide, 2026. [Daring]. Tersedia: https://vuejs.org/guide/introduction.html. [Diakses: 23-Okt-2026].

[3] Axios Contributors, "Getting Started," Axios Docs, 2026. [Daring]. Tersedia: https://axios-http.com/docs/intro. [Diakses: 23-Okt-2026].

[4] GitHub Inc., "About protected branches," GitHub Docs, 2026. [Daring]. Tersedia: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches. [Diakses: 23-Okt-2026].

[5] GitHub Inc., "Understanding GitHub Actions," GitHub Docs, 2026. [Daring]. Tersedia: https://docs.github.com/en/actions/get-started/understand-github-actions. [Diakses: 23-Okt-2026].

[6] Docker Inc., "Multi-stage builds," Docker Docs, 2026. [Daring]. Tersedia: https://docs.docker.com/build/building/multi-stage/. [Diakses: 23-Okt-2026].

[7] Docker Inc., "Docker Compose," Docker Docs, 2026. [Daring]. Tersedia: https://docs.docker.com/compose/. [Diakses: 23-Okt-2026].

[8] K. Dunglas dan kontributor, "FrankenPHP: the modern PHP app server," 2026. [Daring]. Tersedia: https://frankenphp.dev/docs/. [Diakses: 23-Okt-2026].

[9] Nginx Inc., "Beginner's Guide," nginx documentation, 2026. [Daring]. Tersedia: https://nginx.org/en/docs/beginners_guide.html. [Diakses: 23-Okt-2026].